SMA EXPERIMENT NO : 05

CODE:

```
import re

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from googleapiclient.discovery import build

from textblob import TextBlob

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.decomposition import LatentDirichletAllocation

from transformers import pipeline

from wordcloud import WordCloud

import requests

import cv2

from PIL import Image

from io import BytesIO

sns.set_style("whitegrid")

plt.rcParams["figure.figsize"] = (10,6)

# API INPUT

API_KEY = input("AIzaSyDFZViTcfxxjFMr67zLyr82Z7ylElnipKg ")

youtube_link = input(" https://www.youtube.com/watch?v=ua-
CiDNNj30&list=PLWKjhJtqVAblQe2CCWqV4Zy3LY01Z8aF1&index=1")

# -------------------------

# Extract Video ID

# -------------------------

def get_video_id(url):
```

```
    pattern = r"(?:v=|/)([0-9A-Za-z_-]{11})"
    match = re.search(pattern, url)
    if not match:
        raise ValueError("Invalid YouTube link")
    return match.group(1)
video_id = get_video_id(youtube_link)
print("Video ID:", video_id)
youtube = build("youtube", "v3", developerKey=API_KEY)
# -------------------------
# VIDEO METADATA ANALYSIS
# -------------------------
video_req = youtube.videos().list(
    part="snippet,statistics",
    id=video_id
).execute()
video_data = video_req["items"][0]
title = video_data["snippet"]["title"]
description = video_data["snippet"]["description"]
thumbnail_url = video_data["snippet"]["thumbnails"]["high"]["url"]
views = int(video_data["statistics"].get("viewCount",0))
likes = int(video_data["statistics"].get("likeCount",0))
print("\nVideo Title:", title)
print("Views:", views, "Likes:", likes)
# -------------------------
# IMAGE ANALYTICS (Thumbnail)
# -------------------------
print("\nAnalyzing Thumbnail Image...")
```

```
response = requests.get(thumbnail_url)
img = Image.open(BytesIO(response.content))
img_gray = cv2.cvtColor(np.array(img), cv2.COLOR_BGR2GRAY)
brightness = img_gray.mean()
print("Thumbnail Brightness Score:", brightness)
# -------------------------
# COMMENT COLLECTION
# -------------------------
rows = []
next_page = None
while True:
    req = youtube.commentThreads().list(
        part="snippet",
        videoId=video_id,
        maxResults=100,
        pageToken=next_page
    ).execute()
    for item in req["items"]:
        s = item["snippet"]["topLevelComment"]["snippet"]
        rows.append({
            "text": s["textDisplay"],
            "likes": s["likeCount"],
            "time": s["publishedAt"]
        })
    next_page = req.get("nextPageToken")
    if not next_page:
        break
```

```
df = pd.DataFrame(rows)
print("Comments collected:", len(df))
# -------------------------
# TEXT CLEANING
# -------------------------
def clean(t):
    t = re.sub("<.*?>", "", str(t))
    t = re.sub("[^a-zA-Z ]", "", t)
    return t.lower()
df["clean"] = df["text"].apply(clean)
df["length"] = df["clean"].apply(len)
# -------------------------
# SENTIMENT ANALYSIS
# -------------------------
df["polarity"] = df["clean"].apply(
    lambda x: TextBlob(x).sentiment.polarity
)
df["sentiment"] = df["polarity"].apply(
    lambda p: "Positive" if p>0 else ("Negative" if p<0 else "Neutral")
)
print("\nSentiment Counts:")
print(df["sentiment"].value_counts())
# -------------------------
# EMOJI / EMOTICON ANALYSIS
# -------------------------
emoji_pattern = re.compile("[\U00010000-\U0010ffff]", flags=re.UNICODE)
```

```
df["emoji_count"] = df["text"].apply(
    lambda x: len(emoji_pattern.findall(str(x)))
)
print("\nTotal Emojis Used:", df["emoji_count"].sum())
# -------------------------
# EMOTION AI MODEL
# -------------------------
print("\nLoading Emotion AI Model...")
emotion_model = pipeline(
    "text-classification",
    model="j-hartmann/emotion-english-distilroberta-base"
)
df["emotion"] = df["clean"].apply(
    lambda x: emotion_model(x[:200])[0]["label"]
    if len(x)>5 else "neutral"
)
print("\nEmotion Counts:")
print(df["emotion"].value_counts())
# -------------------------
# KEYWORD & TREND ANALYSIS
# -------------------------
vec = CountVectorizer(stop_words="english", max_features=20)
X = vec.fit_transform(df["clean"]
words = vec.get_feature_names_out()
counts = X.sum(axis=0).A1
keyword_df = pd.DataFrame({
    "word":words,
```

```
    "count":counts
}).sort_values("count",ascending=False)
print("\nTop Keywords:")
print(keyword_df.head(10))
# -------------------------
# TOPIC MODELING
# -------------------------
lda = LatentDirichletAllocation(
    n_components=4,
    random_state=0
)
lda.fit(X)
df["topic"] = lda.transform(X).argmax(axis=1
# -------------------------
# BUSINESS ENGAGEMENT SCORE
# -------------------------
df["engagement_score"] = df["likes"]*(df["polarity"]+1)
print("\nBrand Sentiment Score:",
      round(df["polarity"].mean(),3))
# -------------------------
# WORD CLOUD (TREND ANALYSIS)
# -------------------------
text_blob = " ".join(df["clean"]
wc = WordCloud(
    background_color="white",
    width=800,
    height=400
```

```
).generate(text_blob)
plt.imshow(wc)
plt.axis("off")
plt.title("Keyword Trend WordCloud")
plt.show(
# -------------------------
# VISUAL DASHBOARD
# -------------------------
df["replies_cat"] = pd.cut(
    df["likes"],
    bins=[-1,10,100,10000],
    labels=["Low","Medium","High"]
)
fig, axes = plt.subplots(2,3, figsize=(18,10))
fig.suptitle(
    "YouTube Content Based Social Media Analytics Dashboard",
    fontsize=18,
    fontweight="bold"
)
# Sentiment Distribution
sns.countplot(
    ax=axes[0,0],
    data=df,
    x="sentiment",
    palette={"Positive":"green","Neutral":"gray","Negative":"red"}
)
axes[0,0].set_title("1. Sentiment Distribution")
```

```
# Pie Chart
sent_counts = df["sentiment"].value_counts()
axes[0,1].pie(
    sent_counts,
    labels=sent_counts.index,
    autopct="%1.1f%%",
    colors=["gray","green","red"]
)
axes[0,1].set_title("2. Sentiment Percentages")
# Likes vs Sentiment
sns.boxplot(
    ax=axes[0,2],
    data=df,
    x="sentiment",
    y="likes"
)
axes[0,2].set_title("3. Likes by Sentiment"
# Sentiment vs Engagement
heat = pd.crosstab(df["sentiment"], df["replies_cat"])
sns.heatmap(
    heat,
    annot=True,
    fmt="d",
    cmap="Blues",
    ax=axes[1,0]

axes[1,0].set_title("4. Sentiment vs Engagement")
```

```
# Comment Length vs Likes
sns.scatterplot(
    ax=axes[1,1],
    data=df,
    x="length",
    y="likes",
    hue="sentiment"
)
axes[1,1].set_title("5. Likes vs Comment Length")
# Correlation Matrix
corr = df[["likes","length","polarity","engagement_score"]].corr(
sns.heatmap(
    corr,
    annot=True,
    cmap="viridis",
    ax=axes[1,2]
)
axes[1,2].set_title("6. Feature Correlation")
plt.tight_layout(rect=[0,0,1,0.96])
plt.show()
```

OUTPUT:

AIzaSyDFZViTcfxxjFMr67zLyr82Z7ylElnipKg AIzaSyDFZViTcfxxjFMr67zLyr82Z7ylElnipKg

https://www.youtube.com/watch?v=ua-CiDNNj30&list=PLWKjhJtqVAblQe2CCWqV4Zy3LY01Z8aF1&index=1https://www.youtube.com/watch?v=ua-CiDNNj30&list=PLWKjhJtqVAblQe2CCWqV4Zy3LY01Z8aF1&index=1

Video ID: ua-CiDNNj30

Video Title: Learn Data Science Tutorial - Full Course for Beginners

Views: 4368567 Likes: 89428

Analyzing Thumbnail Image...

Thumbnail Brightness Score: 43.04997685185185

Comments collected: 972

Sentiment Counts:

sentiment

Positive 467

Neutral 434

Negative 71

Name: count, dtype: int64

Total Emojis Used: 15

Loading Emotion AI Model...

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:

The secret `HF_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.

You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(

config.json:

1.00k/? [00:00<00:00, 60.0kB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

pytorch_model.bin: 100%

329M/329M [00:02<00:00, 260MB/s]

Loading weights: 100%

105/105 [00:00<00:00, 248.28it/s, Materializing param=roberta.encoder.layer.5.output.dense.weight]

model.safetensors: 100%

329M/329M [00:03<00:00, 152MB/s]

RobertaForSequenceClassification LOAD REPORT from: j-hartmann/emotion-english-distilroberta-base

Key | Status | |

roberta.embeddings.position_ids | UNEXPECTED | |

Notes:

- UNEXPECTED:can be ignored when loading from different task/architecture; not ok if you expect identical arch.

tokenizer_config.json: 100%

294/294 [00:00<00:00, 12.0kB/s]

vocab.json:

798k/? [00:00<00:00, 4.59MB/s]

merges.txt:

456k/? [00:00<00:00, 4.37MB/s]

tokenizer.json:

1.36M/? [00:00<00:00, 7.94MB/s]

special_tokens_map.json: 100%

239/239 [00:00<00:00, 2.02kB/s]

Emotion Counts:

emotion

neutral 404

joy 287

surprise 113

fear 78

sadness 64

anger 21

disgust 5

Name: count, dtype: int64

Top Keywords:

	word	count
2	data	470
13	science	286
17	video	140
15	thank	115
1	course	98
9	like	97
16	thanks	83
5	im	67
6	just	66
4	great	66

Brand Sentiment Score: 0.167

Keyword Trend WordCloud
data science
thank
video
course
data
great
good
much
watching
work
amazing
make
hour
dont
tutorial
learn
need
one
skill
job
field
python
time
start
im
best
know
will
coding
understand
people
overview
statistic
please
content
data scientist
watch
think
anyone
free
really
help
lot
still
math
way
guy
want
beginner
business
now
thing
freecodecamp
student
explanation
awesome
learning
language
year
part
tool
use
stuff
looking
day
minute
knowledge
starting
example
background
excellent
many
sir
hello
model
better
even
nice
put
cant
crucial
team
material
life
teaching
information
understanding
something
say
making
easy
made
statistical
analytics
programming
comment
helpful
different
data analysis
feel
code
coding statistics
mean
someone
finished
download
database
machine learning
effort
concept
hey
channel
done
provide
sure
excel
using
youtube
unicorn
everything
started
regression
method
probability
speed
question
big data
degree
hi
steps
part
x
level
wow
fit
everyone
first
class
research
working
useful
lecture
possible
problem
give
new
live
master
voice
getting
see
become
share
clear
engineer
real
without
hypothesis testing
topic
kind
far
two
used
minute
example
knowledge
starting
stuff
looking
day
best
job
use
start
time
python
learning
language
year
part
tool
journey
career
value
involve
program
simple
yet
goal
web
dont
file
full
teach
got
hope
didnt
valuable
u
barton poulson
introduction
next step
already
go
project
experience
analysis
basic
insight
less
without
coding statistics
mean
someone
finished
download
database
machine learning
effort
concept
hey
channel
done
provide
sure
excel
using
helpful
different
data analysis
feel
code
comment
doesnt
programming